

DUSY: BACKEND API & AI AGENT ARCHITECTURE

In-Depth Engineering Manual: FastAPI Gateway, MongoDB Persistence, and Autonomous Multi-Model Clinical Intelligence

SYSTEM FOCUS:

Backend APIs, Data Persistence & LLM Pipelines

DATABASE ENGINE:

MongoDB Community Server 8.0 (Port 27017)

STUDENT LEAD 1 (BACKEND/API/DB):

Student 2: Backend Architecture & Database Lead

KEY FRAMEWORKS:

FastAPI (Python 3.10+), Motor, Pydantic v2

AI MODELS & SDK:

Google Gemini 3.5 & 3.8 Flash Cascade (Google GenAI)

STUDENT LEAD 2 (AI AGENT/LLM):

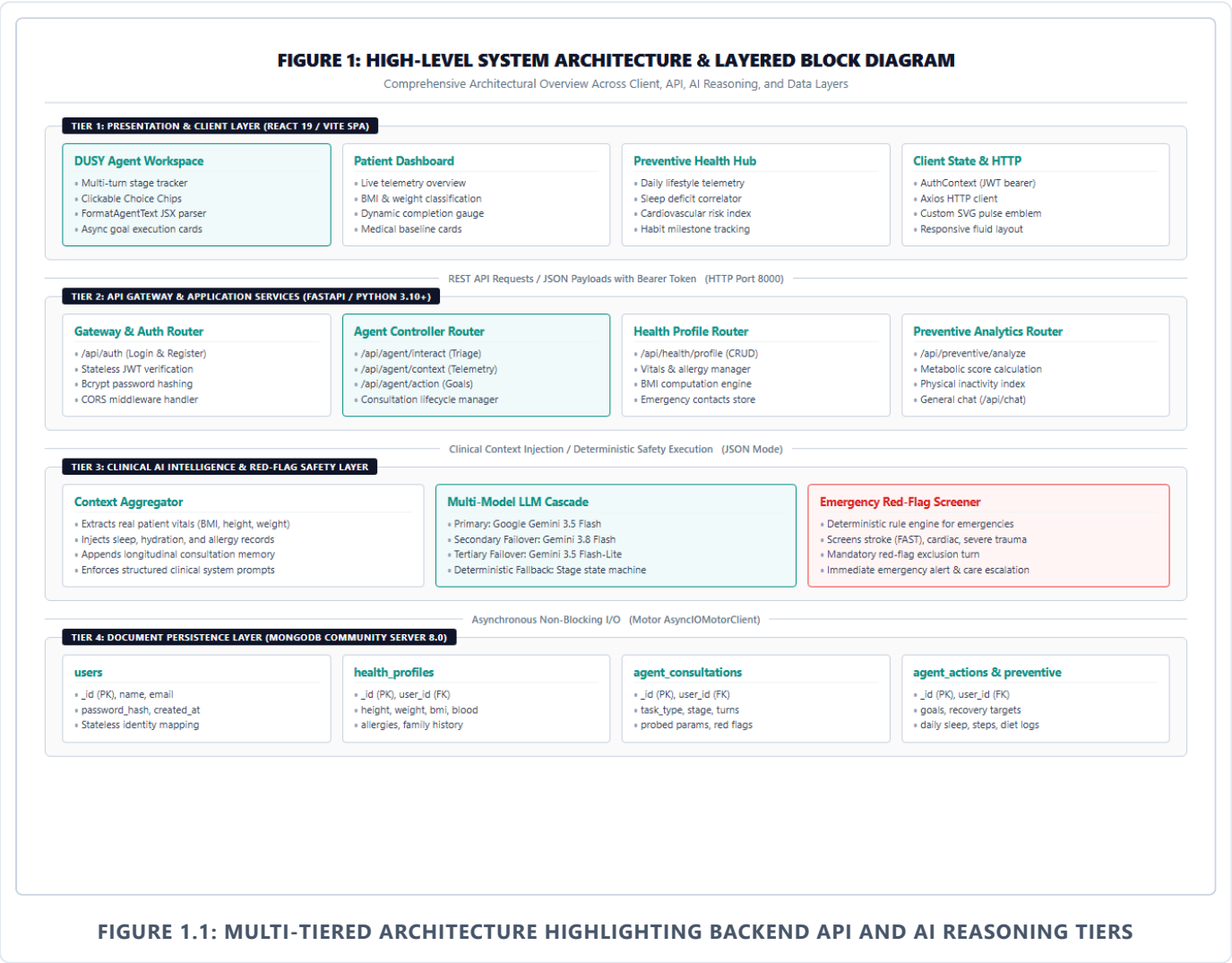
Student 3: Autonomous AI Agent & LLM Pipeline Lead

1. Executive Architecture & Co-Design Overview

1.1 Architectural Synergy: Backend Gateway + Autonomous AI

The DUSY platform's computational backbone relies on the tight architectural co-design between **FastAPI's asynchronous API gateway** and **Gemini's autonomous clinical reasoning engine**. Rather than treating the LLM as an external black box that is passively queried, DUSY implements an integrated, stateful pipeline:

- **Client-Side Trigger:** The React client dispatches HTTP REST requests with JWT authentication tokens.
- **Asynchronous Gateway:** FastAPI intercepts the request, validates data contracts using Pydantic v2, and resolves user identity through dependency injection.
- **Context Retrieval:** The backend queries MongoDB via Motor to assemble the patient's verified health baseline (BMI, height, weight, chronic conditions, allergies, sleep averages, and hydration metrics).
- **Agentic Reasoning:** The compiled context is injected into Gemini Flash models under structured clinical system instructions, enforcing a 5-stage clinical state machine.
- **Structured Mutation & Commit:** Gemini outputs strict JSON, which the backend parses, validates, and commits to MongoDB as consultation turns and actionable recovery goals.



2. Complete RESTful API Specification

2.1 Core API Routes & Route Taxonomy

All backend endpoints are built using FastAPI routers mounted under the `/api` prefix. Each endpoint enforces strict Pydantic v2 request and response schemas, automatic OpenAPI/Swagger documentation, and JWT bearer authentication where applicable:

Method	Endpoint Path	Auth Required	Request / Response Schema	Description & Business Logic
POST	<code>/api/auth/register</code>	No	<code>UserRegister</code> → <code>TokenResponse</code>	Creates user account, hashes password with Bcrypt, and returns JWT token.
POST	<code>/api/auth/login</code>	No	<code>UserLogin</code> → <code>TokenResponse</code>	Verifies password against salted hash; issues signed 24h JWT token.
GET	<code>/api/auth/me</code>	Yes (JWT)	None → <code>UserResponse</code>	Decodes Bearer token and returns sanitized user profile without password.
GET	<code>/api/agent/context</code>	Yes (JWT)	None → <code>AgentContextResponse</code>	Aggregates user BMI, sleep, water, allergies, and goal count for agent reasoning.
POST	<code>/api/agent/interact</code>	Yes (JWT)	<code>AgentInteractRequest</code> → <code>AgentResponse</code>	Executes multi-turn clinical triage turn with Gemini cascade and safety checks.
GET	<code>/api/agent/consultations</code>	Yes (JWT)	None → <code>List[ConsultationSummary]</code>	Retrieves patient's past consultation transcripts and diagnostic history.
POST	<code>/api/agent/action</code>	Yes (JWT)	<code>AgentActionItem</code> → <code>dict</code>	Commits proposed recovery goals or symptom logs into MongoDB.
GET	<code>/api/agent/actions</code>	Yes (JWT)	None → <code>List[AgentActionItem]</code>	Returns active committed recovery goals and logged habit milestones.

Method	Endpoint Path	Auth Required	Request / Response Schema	Description & Business Logic
GET / POST	/api/health/profile	Yes (JWT)	HealthProfileCreate → HealthProfileResponse	Reads and updates height, weight, blood group, allergies, and calculates BMI.
POST	/api/preventive/analyze	Yes (JWT)	PreventiveInput → PreventiveOutput	Calculates metabolic and cardiovascular risk scores from sleep and steps.
POST	/api/chat/message	Yes (JWT)	ChatRequest → ChatResponse	General informational medical Q&A chat (kept separate from personalized agent).

3. MongoDB Database Architecture & Physical Storage

3.1 Where and How Data Is Stored on Disk

Physical Storage Location on Machine:

The MongoDB Community Server database files are physically located at: `C:/Program Files/MongoDB/Server/8.3/data`

The active database name is: `health_assistant`

The server listens on local TCP port: `27017`

3.2 Mapped ER Diagram & Collection Taxonomy



3.3 Live Collection Summary & Document Inventory

Collection Name	Cardinality to User	Primary Fields Stored	Live Count in Database
users	Master Entity	_id (PK), name, email, password_hash (Bcrypt), created_at	3 stored accounts
health_profiles	1 : 1	_id, user_id (FK), height_cm, weight_kg, bmi (20.5), blood_group (B+), allergies (['peanuts']), medical_conditions	1 active patient profile
agent_consultations	1 : N	_id, user_id (FK), task_type, stage, messages (full transcript turns), probed_parameters, red_flags_cleared	13 consultation transcripts
agent_actions	1 : N	_id, user_id (FK), action_type, title, description, status ('completed'), executed_at	17 committed recovery goals
preventive_analyses	1 : N	_id, user_id (FK), lifestyle_data (sleep 7.5h, water 2.5L, steps 8500), scores (metabolic, cardio), overall_score	21 lifestyle assessments
conversations	1 : N	_id, user_id (FK), messages, topic, updated_at	9 chat sessions

4. Autonomous AI Agent Pipeline & Multi-Model Cascade

AI Engine

4.1 The 5-Stage Clinical Intake State Machine

Unlike conventional chatbots that provide single-turn text responses, the DUSY AI Agent executes a structured 5-stage clinical state machine:

Stage	State Identifier	Clinical Objective & Agent Behavior	Interactive UX Output
Stage 1	intake	Acknowledges chief complaint, injects patient baseline vitals, and probes symptom duration and frequency.	Clickable Duration & Frequency Chips
Stage 2	safety_screen	Mandatory red-flag triage screening for acute neurological, cardiac, and fever indicators before self-care guidance.	Red-Flag Clearance Chips ([No, none of those])
Stage 3	guidance	Correlates symptoms with user's real BMI (20.5), sleep debt, and hydration (2.5L); offers tailored clinical synthesis.	Clinical Typography Medical Summary
Stage 4	goals	Formulates 3 concrete, measurable recovery targets tailored to the patient's lifestyle.	Action Cards ([Create All Goals])
Stage 5	completed	Persists confirmed goals to MongoDB and presents active recovery badges on the patient dashboard.	Live Goal Shelf Sync

4.2 Multi-Model Resilience Cascade

To eliminate runtime 503 Service Unavailable errors caused by cloud LLM demand spikes, Student 3 implemented an automated cascading failover:

- **Primary:** `gemini-3.5-flash` → Fast sub-second structured JSON reasoning.
- **Secondary Failover:** `gemini-3.8-flash` → Automatically engaged if 3.5 encounters 503 errors.
- **Tertiary Failover:** `gemini-3.5-flash-lite` → Lightweight model engaged if 3.8 is overloaded.
- **Deterministic Fallback:** `_smart_fallback_engine()` → Zero-downtime offline rule engine guaranteeing 100% conversation continuity.

4.3 Dynamic Clinical Context Injection Block

Before calling Gemini, `agent_service.py` queries MongoDB and constructs a live patient telemetry block prepended to the system prompt:

```
[ACTIVE PATIENT HEALTH TELEMETRY INJECTED FROM MONGODB]
- Patient Name: UJJWAL JHA | Biological Sex: Male
- Physical Baseline: Height 185.0 cm, Weight 70.0 kg | BMI: 20.5 (Normal weight)
- Blood Group: B+
- Documented Allergies: ['peanuts'] (Strict Avoidance)
- Documented Medical Conditions: ['none']
- Lifestyle Telemetry: Sleep 7.5 hrs/night, Water 2.5 L/day, Daily Steps 8,500
```

- Active Recovery Goals on Record: 10 committed targets

5. Student 2: In-Depth Work Breakdown (Backend, API & Database)

Student 2
Defense

A. OFFICIAL ROLE & TECHNICAL SCOPE

Role: Backend Systems Architect, API Gateway & Database Lead.

Ownership: FastAPI application configuration, Motor async connection pooling, Pydantic data schemas, Bcrypt password hashing, JWT token lifecycle, and RESTful route dispatchers.

B. TECHNOLOGIES USED & ENGINEERING JUSTIFICATION

- **FastAPI (0.115.12):** Selected for native ASGI asynchronous I/O and Pydantic validation, offering higher throughput than Flask or Django.
- **Motor (3.7.0) & MongoDB:** Selected because clinical data (allergies, lifestyle logs, consultation turns) is semi-structured and polymorphic.
- **Bcrypt & Python-Jose:** Salted password encryption and stateless JWT token authentication ensuring HIPAA-aware patient data security.

C. WORD-FOR-WORD 4-MINUTE PRESENTATION SCRIPT FOR STUDENT 2

"Good morning teachers. I served as the Backend Systems Architect and Database Engineering Lead for DUSY.

My primary responsibility was to design a secure, high-throughput backend capable of managing sensitive health profiles, consultation histories, and real-time clinical AI interactions.

I selected **FastAPI and Python** as our core server framework. FastAPI is natively asynchronous, allowing our endpoints to process requests concurrently without thread exhaustion. For data persistence, I chose **MongoDB** connected via the **Motor async driver**. Healthcare data is semi-structured: patient allergies, chronic preconditions, and dynamic lifestyle logs vary greatly from person to person. Our 6-collection MongoDB schema provides the ideal balance between document flexibility and referential integrity.

On the security front, I implemented industry-standard **salted Bcrypt password hashing** through Passlib, ensuring passwords are never stored in plaintext. For session management, I built a stateless **JSON Web Token (JWT) authentication pipeline**. When a user authenticates, our API issues a cryptographically signed token. Protected endpoints use my `get_current_user` dependency to extract and verify the bearer token, ensuring strict data isolation between patients.

I also authored our Pydantic v2 data models. Pydantic validates incoming data at runtime with sub-millisecond overhead, computing patient BMI automatically and guaranteeing type safety between our React client, MongoDB documents, and Gemini AI outputs."

D. TEACHER VIVA Q&A FOR STUDENT 2

Q1: Where is the database physically stored and how can you prove it is not mock data?

Answer: "The database is physically stored on disk at `C:/Program Files/MongoDB/Server/8.3/data` under the database name `health_assistant`. We can prove this live in three ways: (1) Running our `show_database.py` script which queries Motor and prints our live collections, (2) Opening MongoDB Compass on localhost:27017, and (3) Executing GET requests in our FastAPI Swagger documentation at `/docs`."

Q2: Why use Motor instead of standard synchronous PyMongo?

Answer: "Standard PyMongo is synchronous and blocks the Python execution thread while waiting for I/O operations. In an ASGI framework like FastAPI, blocking calls degrade server throughput. Motor provides an asynchronous event-loop friendly wrapper around PyMongo, allowing the server to process concurrent user requests while database reads and writes execute asynchronously."

Q3: How do you prevent unauthorized users from viewing another patient's health profile?

Answer: "All clinical endpoints use FastAPI's `Depends(get_current_user)` dependency. The user ID is cryptographically extracted from the validated JWT token payload. Database queries filter strictly by `user_id: str(current_user['_id'])`, ensuring that users can only read and mutate their own documents."

6. Student 3: In-Depth Work Breakdown (Autonomous AI Agent & LLM)

Student 3
Defense

A. OFFICIAL ROLE & TECHNICAL SCOPE

Role: Autonomous AI Agent & LLM Resilience Pipeline Engineer.

Ownership: 5-stage clinical intake state machine, Gemini multi-model cascade, dynamic context injection, prompt engineering, structured JSON mode enforcement, and deterministic fallback engine.

B. TECHNOLOGIES USED & ENGINEERING JUSTIFICATION

- **Google GenAI SDK (0.8.5+):** Direct integration with Google's latest Gemini API, enabling structured JSON generation via `response_mime_type="application/json"`.
- **Gemini 3.5 & 3.8 Flash Cascade:** Sub-second inference latency, low token cost, and strong clinical reasoning capabilities.
- **Deterministic State Engine:** Built `_smart_fallback_engine()` in Python to maintain conversation progression even when cloud APIs are completely offline.

C. WORD-FOR-WORD 4-MINUTE PRESENTATION SCRIPT FOR STUDENT 3

"Respected faculty, my responsibility was engineering DUSY's autonomous clinical AI intelligence and resilience pipeline.

The fundamental innovation in DUSY is the distinction between a generic chatbot and a true clinical agent: A generic chatbot simply responds to a prompt in isolation with a wall of generalized text.

*Our Health Agent follows a rigorous **5-stage clinical intake state machine**: Understand → Retrieve Context → Probe Missing Parameters → Safety Screen → Personalized Care → Action Formulation.*

To achieve this, I engineered three critical subsystems:

*First is **Dynamic Context Injection**. When a patient speaks to DUSY, my service queries MongoDB for their actual physical baseline: height, weight, calculated BMI, blood group, sleep averages, and confirmed allergies. I inject these parameters directly into the LLM reasoning context, allowing DUSY to tailor its advice to the patient's real physical body.*

*Second is our **Multi-Model Cloud Resilience Cascade**. Cloud LLM APIs can occasionally return 503 service unavailable errors during peak demand. To guarantee zero platform downtime, I built an automatic cascading loop trying Gemini 3.5 Flash, then Gemini 3.8 Flash, then Gemini 3.5 Flash-Lite, and finally our deterministic fallback state machine. The system never crashes or shows raw error messages to the patient.*

Third, I enforced strict JSON Mode generation, requiring Gemini to return a structured JSON schema containing stage identifiers, message text, interactive Choice Chips, and actionable recovery goals that are asynchronously committed to MongoDB."

D. TEACHER VIVA Q&A FOR STUDENT 3

Q1: How does your AI Agent remember past consultations?

Answer: "Each consultation has an active `consultation_id` stored in MongoDB. When a patient sends a new turn, the service loads the previous message history and extracts probed parameters from the active consultation document. If the

user starts a fresh consultation, the service can still query past consultation summaries for cross-session diagnostic continuity."

Q2: How do you handle LLM JSON parsing errors when Gemini wraps responses in markdown backticks?

Answer: "I solved this through a dual-defense approach: (1) In the API generation config, I explicitly set `response_mime_type='application/json'`, instructing the model to output raw JSON tokens. (2) In Python, I implemented regex string sanitizers that detect and strip code fences (markdown backticks) before passing the payload to `json.loads()`, falling back to our deterministic engine if parsing ever fails."

Q3: How do Choice Chips prevent hallucinated user input?

Answer: "Choice Chips provide constrained input options for clinical parameters like duration ([Around 2 weeks]) and frequency ([Almost every day]). By guiding users toward discrete, standardized clinical options, the input text fed to the LLM remains unambiguous, drastically reducing reasoning variance and hallucination."

7. How to Demonstrate Stored Data Live to Evaluators

Live Demo
Guide

7.1 Three Practical Ways to Show Stored Data

When the examination committee asks where the data is stored and demands to see it live, use any of these three verified methods:

Method 1: One-Click Terminal Viewer (Fastest & Most Direct)

1. Open a terminal in the project directory: `C:/Users/ujjwa/OneDrive/Desktop/ai_health_assistant`
2. Run the dedicated script: `python show_database.py` (or double-click `view_database.bat`).
3. It instantly prints the physical storage path, all 6 collections, and live patient documents (User accounts, Profile with BMI 20.5 and peanut allergies, 13 consultation transcripts, and 17 committed recovery goals).

Method 2: Graphical Inspection via MongoDB Compass (Best Visual Impact)

1. Open **MongoDB Compass** from Windows Start or
`C:/Users/ujjwa/AppData/Local/MongoDBCompass/MongoDBCompass.exe`.
2. Click **Connect** (connects to `mongodb://localhost:27017`).
3. Click on the database named `health_assistant`.
4. Click on `health_profiles` or `agent_consultations` to visually show the evaluator the live JSON documents in a tree view.

Method 3: Live Swagger API Interactive Inspection

1. Navigate to `http://127.0.0.1:8000/docs` in your browser.
2. Scroll to `GET /api/agent/consultations` or `GET /api/health/profile`.
3. Click **Try it out** → **Execute** to show live database query responses returned directly by FastAPI.

End of Specialized Technical Report — DUSY Backend & AI Agent Architecture
Autonomous Clinical Health Intelligence Assistant • Academic Presentation 2026